

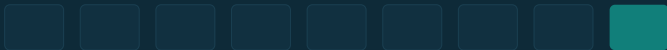


STREAM PROCESSING AVEC APACHE KAFKA

Sémantiques de livraison & exactly-once

Idempotence · transactions · consume-transform-produce

ESGI · 5^e année · Data Engineering · 2025–2026



SÉANCE 6

Objectifs

- **Distinguer les trois sémantiques**

at-most-once, at-least-once, exactly-once — d'où viennent pertes et doublons.

- **Comprendre l'idempotence du producteur**

PID + numéros de séquence, activée par défaut depuis Kafka 3.0.

- **Mettre en œuvre les transactions**

Écriture atomique multi-partitions et pattern *consume-transform-produce*.

- **Lire proprement**

`isolation.level=read_committed` : ne voir que les messages validés.

SÉANCE 6

Au programme

01

Trois sémantiques

perte vs doublon

02

Idempotence producteur

PID + séquence

03

Transactions & EOS

atomique,
read_committed

04

Atelier pipeline EOS

consume-transform-
produce

01

- PARTIE 1

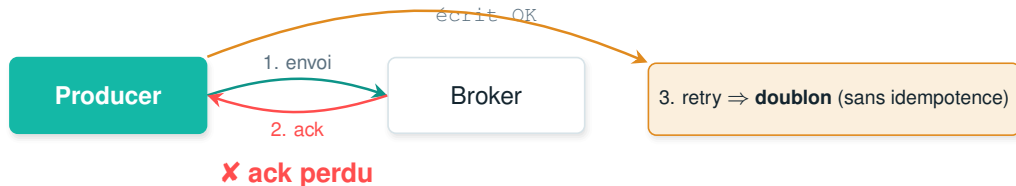
Les sémantiques de livraison

Les trois sémantiques

Sémantique	D'où ça vient	Risque
at-most-once	<code>acks=0</code> , ou commit <i>avant</i> traitement	perte
at-least-once	retry du producteur, ou commit <i>après</i> traitement	doublons
exactly-once	idempotence + transactions	aucun (dans Kafka)

L'*exactly-once* de Kafka vaut à l'intérieur d'un **cluster Kafka** — pas vers un système externe (base, API).

D'où viennent les doublons



■ Côté producteur

Un ack perdu déclenche un retry $\Rightarrow$ le message est écrit deux fois.

■ Côté consommateur

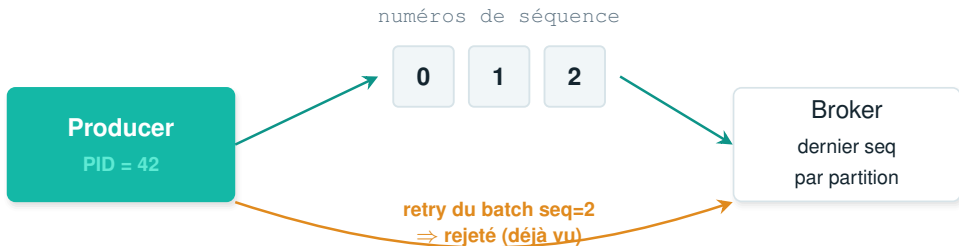
Commit *avant* traitement $\Rightarrow$ perte ; *après* $\Rightarrow$ doublon (rappel S5).

02

- PARTIE 2

Idempotence du producteur

Le producteur ne se répète pas



- Le broker attribue un **PID** par producteur et suit le **numéro de séquence** par partition : un batch déjà vu est rejeté (dédoublonnage + ordre).
- **Activée par défaut depuis Kafka 3.0** : `enable.idempotence=true` avec `acks=all` (et `max.in.flight ≤ 5`).

Voir le PID & la séquence dans le log

Produire, puis inspecter le segment du topic

```
# Produire (producteur idempotent par défaut)
kafka-console-producer.sh --bootstrap-server localhost:29092 --topic orders
> commande-1      > commande-2
# Inspecter le segment : producerId, producerEpoch, sequence
docker compose exec kafka-1 /opt/kafka/bin/kafka-dump-log.sh \
  --files /var/lib/kafka/data/orders-0/00000000000000000000.log \
  --print-data-log
```

■ Dans la sortie

producerId, producerEpoch,
baseSequence par batch.

■ À retenir

Base du dédoublement : aucune config à
activer en 4.x.

03

- PARTIE 3

Transactions & exactly-once

PARTIE 3 · TRANSACTIONS

Tout ou rien, sur plusieurs partitions

- Un `transactional.id` unique et stable identifie le producteur ; il **implique l'idempotence** (`enable.idempotence=true`).
- Écriture **atomique** sur plusieurs partitions / topics : tout est validé, ou rien n'est visible.
- Géré par un *transaction coordinator* via le topic interne `__transaction_state` ; les zombies sont exclus par *epoch fencing*.

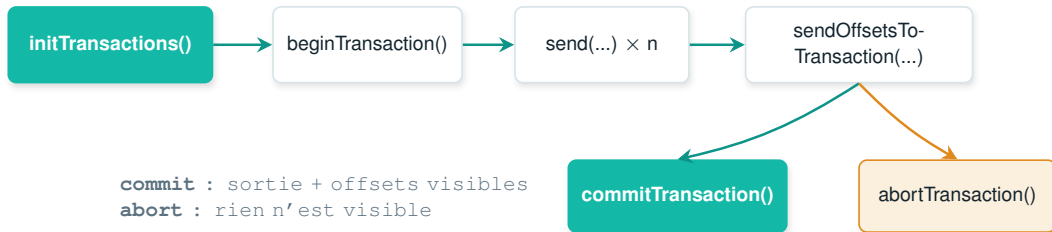
■ Idempotence

Pas de doublon de retry **sur une** partition.

■ Transactions

Atomicité **entre** partitions, topics *et* offsets.

Une transaction, étape par étape



`initTransactions()` une seule fois au démarrage ; puis `begin` → `send` → `sendOffsets` → `commit/abort` à *chaque lot*.

Lire validé, traiter, reproduire

Coeur du pipeline exactly-once (consume-transform-produce)

```
producer.initTransactions(); // une fois
while (running) {
    var records = consumer.poll(Duration.ofMillis(500));
    producer.beginTransaction();
    try {
        for (var r : records)
            producer.send(toValidated(r)); // transform + cle -> sortie
        producer.sendOffsetsToTransaction(
            offsets(records), consumer.groupMetadata());
        producer.commitTransaction(); // sortie + offsets atomiques
    } catch (Exception e) {
        producer.abortTransaction(); // rien n'est visible
    }
}
```

Abort vs read_committed

Comparer ce que voit un consommateur selon l isolation

```
# Ne voir QUE les messages committes (filtre les transactions abandonnees)
kafka-console-consumer.sh --bootstrap-server localhost:29092 \
  --topic orders-validated --from-beginning \
  --isolation-level read_committed
```

```
# Tout voir, y compris une transaction abandonnee
kafka-console-consumer.sh --bootstrap-server localhost:29092 \
  --topic orders-validated --from-beginning \
  --isolation-level read_uncommitted
```

À observer : faites abandonner une transaction (message empoisonné dans l'atelier). En `read_committed`, ces messages **n'apparaissent pas** ; en `read_uncommitted`, ils apparaissent.

04

- PARTIE 4

Atelier — pipeline exactly-once

Les topics du pipeline

Demarrer le cluster et creer les deux topics

```
docker compose up -d                                # 3 brokers KRaft

kafka-topics.sh --bootstrap-server localhost:29092 \
  --create --topic orders --partitions 3 --replication-factor 3
kafka-topics.sh --bootstrap-server localhost:29092 \
  --create --topic orders-validated --partitions 3 --replication-factor 3

# Alimenter le topic d'entree (seeder fourni dans le kit)
mvn -q compile exec:java -Dexec.mainClass=fr.esgi.kafka.tp6.OrderSeeder
```

■ Entrée / sortie

orders → traitement transactionnel →

• ESGI · Stream Processing avec Apache Kafka
orders-validated.

■ Réplication 3

Coordinateur de transactions fiable :

__transaction_state en RF 3.

Séance 6 · 16

Construire le pipeline EOS

- 1 Configurer le consommateur : `enable.auto.commit=false` + `read_committed`.
- 2 Configurer le producteur transactionnel : `transactional.id`, puis `initTransactions()`.
- 3 Compléter la boucle : `begin` → `send` vers `orders-validated` → `sendOffsets` → `commit`.
- 4 Sur un message empoisonné, `abortTransaction()` au lieu de `commit`.
- 5 Lancer le pipeline, puis lire `orders-validated` en `read_committed`.

Vérifier l'exactly-once

Compter ce qui est réellement valide

```
# Nombre de messages VALIDES en sortie (transactions committed)
kafka-console-consumer.sh --bootstrap-server localhost:29092 \
  --topic orders-validated --from-beginning \
  --isolation-level read_committed --timeout-ms 4000 | wc -l

# Progression atomique des offsets du groupe consommateur
kafka-consumer-groups.sh --bootstrap-server localhost:29092 \
  --describe --group eos-pipeline
```

■ Pas de doublon

Même en relançant le pipeline, le décompte
en `read_committed` reste stable.

■ Offsets liés

Les offsets n'avancent que si la transaction
est validée.

Pour aller plus loin

Objectif — prouver l'exactly-once de bout en bout malgré un crash, et mesurer le coût des transactions.

■ 1. Crash test

Tuer le pipeline en plein lot ; vérifier qu'aucun demi-résultat n'est visible.

■ 2. Coût

Comparer le débit avec et sans transactions (commit par lot vs par message).

■ 3. Bonus

Faire varier la taille des lots et observer l'effet latence / débit.

Share groups — Queues for Kafka (KIP-932)

- Un **nouveau type de groupe** (à côté de `classic` et `consumer`) : les consommateurs se partagent une partition **sans** exclusivité — donc **consos** > **partitions**.
- **Acquittement par message** (ACCEPT / RELEASE / REJECT) + comptage des tentatives ; sémantique de *file d'attente*.
- **GA en Kafka 4.2** ; côté outils, `kafka-console-share-consumer.sh` et l'API `KafkaShareConsumer`.

■ Consumer group

Ordre par partition, 1 conso/partition, parallélisme borné.

■ Share group

Travail réparti par message, conso élastiques, pas d'ordre garanti.

SÉANCE 6

À retenir

- ✓ Trois sémantiques : at-most-once (perte), at-least-once (doublons), exactly-once.
- ✓ L'idempotence (PID + séquence) supprime les doublons de retry, **par défaut depuis 3.0**.
- ✓ Les transactions rendent atomiques la sortie *et* les offsets, sur plusieurs partitions.
- ✓ `read_committed` : on ne lit que ce qui est validé (jusqu'au LSO).
- ✓ `consume-transform-produce` = le pattern exactly-once *à l'intérieur* de Kafka.



LA SUITE — SÉANCE 7

Sérialisation & Schema Registry

Avro et Protobuf, registre de schémas, et évolution compatible des formats de messages.